SEASUG Paper 50-2026

Understanding SAS DATA Step Compilation, Execution, and _n_ Using Examples

Kiran Venna

ABSTRACT

Understanding how the SAS DATA step works internally is one of the most important skills for every SAS programmer. This paper focuses on two fundamental concepts: the DATA step compilation phase and execution phase, and the automatic variable _n_. During the compilation phase, the Program Data Vector is created, and variable attributes are defined, while during the execution phase, SAS processes observations by reading, assigning, and writing values to the output dataset. Examples involving single and multiple input datasets are used to demonstrate how DATA step compilation and execution phases affect the final output. The paper also explores that the automatic variable _N_, created during the DATA step, represents an iteration count rather than an observation number, and an example is provided to illustrate the same.

INTRODUCTION

In this paper, we would like to discuss two important concepts of the DATA step. The first is what happens in the compilation and execution phases of the DATA step, how the difference in timing and the way each phase operates affect the output, and what problems we usually face. Understanding the timing and activities performed in each phase can help us avoid these problems. We briefly discuss the functionality of each phase, with several examples showing how each one is performed and what impact it has on the output.

The second concept is the automatic variable _N_, which is the iteration count of the DATA step but is sometimes mistaken for the observation number. A DATA step is used as an example to show how _N_, can differ for the same output. Once the difference is clear, we also discuss how _N_ behaves when it is used in our code.

Concept One: DATA step compilation phase and execution phase.

This discussion of the compilation phase and execution phase will stay at a high level, just enough to understand the concepts covered in this paper. When you run DATA step

code, two important phases occur before you see any output: the **compilation phase** and the **execution phase**.

The compilation phase happens first. It checks syntax and sets variable attributes such as length and data type. Once compilation is complete, the execution phase begins, where data is read, whether from a file, a dataset, a hardcoded value, or any other source.

Now let's look more closely at why a deeper understanding of the compilation phase matters. We'll cover two scenarios: one involving a single dataset, and another involving multiple datasets. In each, we'll walk through a common challenge and show how understanding compilation behavior helps solve it.

Scenario 1: Single dataset

In the example below, we create a new variable, Gender, based on the existing Sex variable in sashelp.class dataset:

```
data class;
set sashelp.class;
if sex='M' then
Gender='Male';
if sex='F' then
Gender='Female';
run;
```

Running a PROC PRINT on the result shows something unexpected, as seen in the partial results below:

```
proc print data=class;

run;
```

Obs	Name	Sex	Age	Height	Weight	Gender
1	Alfred	M	14	69.0	112.5	Male
2	Alice	F	13	56.5	84.0	Fema
3	Barbara	F	13	65.3	98.0	Fema
4	Carol	F	14	62.8	102.5	Fema
5	Henry	M	14	63.5	102.5	Male
6	James	M	12	57.3	83.0	Male
7	Jane	F	12	59.8	84.5	Fema
8	Janet	F	15	62.5	112.5	Fema
9	Jeffrey	M	13	62.5	84.0	Male

The output shows that the values in the new Gender variable are truncated. To understand why, we need to revisit what happens during the two phases, specifically how SAS handles compilation.

During compilation, all variables except newly created ones inherit their attributes directly from the variables named in the SET statement. For a new variable like Gender, SAS scans the code and sets its attributes based on the first instance it encounters. Here, that's Gender='Male', so SAS sets the length to 4. SAS does not revisit the length after that first instance; every later assignment is ignored for attribute-setting purposes.

In the execution phase, when the data is actually read and written, Gender can only hold 4 characters, since that's the length set at compile time. This causes truncation whenever the value is longer than 4 characters, as happens with 'Female'.

We can confirm this with PROC CONTENTS procedure:

```
proc contents data=class;

run;
```

Alphabetic List of Variables and Attributes			
#	Variable	Type	Len
3	Age	Num	8
6	Gender	Char	4
4	Height	Num	8
1	Name	Char	8
2	Sex	Char	1
5	Weight	Num	8

The output confirms that the length of Gender was set to 4.

How do we fix this? There are two approaches:

1. Reorder the IF statements so the longest value ('Female') is assigned first, letting compilation pick up the longer length.
2. Explicitly declare the length with a LENGTH statement.

Once the correct length is set during compilation, execution reads and writes the data accurately, with no truncation.

Approach 1 (highlighted): Reorder so the longest value comes first

```
data Class1;
set sashelp.class;
if sex='F' then
Gender='Female';
if sex='M' then
Gender='Male';
run;
```

Approach 2 (highlighted): Explicitly declare the length

```
data Class1;
set sashelp.class;
length Gender $6;
if sex='M' then
Gender='Male';
if sex='F' then
Gender='Female';
```

```
run;
```

With either fix, PROC PRINT no longer shows truncation. A proc print statment and results (partial results) are shown below.

```
proc print data=class1;
run;
```

Obs	Name	Sex	Age	Height	Weight	Gender
1	Alfred	M	14	69.0	112.5	Male
2	Alice	F	13	56.5	84.0	Female
3	Barbara	F	13	65.3	98.0	Female
4	Carol	F	14	62.8	102.5	Female
5	Henry	M	14	63.5	102.5	Male
6	James	M	12	57.3	83.0	Male
7	Jane	F	12	59.8	84.5	Female
8	Janet	F	15	62.5	112.5	Female
9	Jeffrey	M	13	62.5	84.0	Male

PROC CONTENTS confirms the corrected length and code for Proc contents and results (partial) are shown below.

```
proc contents data=class1;
run;
```

Alphabetic List of Variables and Attributes			
#	Variable	Type	Len
3	Age	Num	8
6	Gender	Char	6
4	Height	Num	8
1	Name	Char	8
2	Sex	Char	1
5	Weight	Num	8

By understanding how the compilation phase sets variable attributes, we can ensure the execution phase reads and writes data precisely, with no unintended truncation.

Now that we've covered the single-dataset case and seen how the compilation and execution phases interact, let's turn to scenarios involving multiple datasets.

Scenario 2: Multiple Datasets

With multiple datasets, two distinct scenarios can occur. The first is like what we saw with a single dataset: compilation impacts execution, and the issue is tied to variable attributes. This shows specifically when multiple datasets are combined in a SET statement. The second scenario is more about how execution itself is affected when multiple datasets share the same variable, in this case, the issue is about data being overwritten.

Scenario 2a: Multiple Datasets , Impact of Compilation on Execution

Let's use two sample datasets to illustrate this concept. Both contain a single variable, Var, but with different lengths:

```
data test1;

length var $3;
input var;
datalines;
aks
;
run;
data test2;
length var $10;
input var;
datalines;
chocholate
;
run;
```

As shown above, Var has a length of 3 in test1 and a length of 10 in test2. If we append test1 and test2 by listing test1 first in the SET statement to create a new dataset, test3, the log returns a warning:

```
data test3;
```

```
SET test1 test2;
run;

WARNING: Multiple lengths were specified for the variable var by input
data set(s). This can cause truncation of data.
```

Let's check the results with a quick PROC PRINT:

```
proc print data=test3;
run;
```

Obs	var
1	aks
2	cho

The output confirms that the value from test2 ("Chocolate") has been truncated. This happens because test1, the first dataset in the SET statement, defines Var with a length of 3 during compilation. SAS sets the variable's length based on that first occurrence and ignores any subsequent length specifications.

Fixing the issue

There are two ways to resolve this:

Approach 1 (highlighted): Reorder the datasets so the variable with the longest length comes first

Since test2 has the longer length, we list it first in the SET statement:

```
data test3_modified;
SET test2 test1;
run;
```

Approach 2 (highlighted): Explicitly declare the length with a LENGTH statement before the SET statement

```
data test3_modified;
```

```
length var $10;
SET test1 test2;
run;
```

Running PROC PRINT on the corrected dataset shows no truncation:

```
proc print data=test3_modified;
run;
```

Obs	var
1	aks
2	chocholate

This is because the longer length is now established before the data is read, either by reordering the datasets or by declaring the length explicitly. We've skipped PROC CONTENTS here since we covered that in detail earlier and want to avoid repetition.

Scenario 2b: Multiple Datasets, Impact of order of Execution on overwriting data

Let's use two sample datasets to illustrate this concept. Both datasets contain two variables, id and name, with the same attributes:

```
data one;
input id name $;
datalines;
1 Robert
;
run;
data two;
input id name $ age;
datalines;
1 Roberd 15
;
run;
```

Now let's merge these datasets by the id variable, as shown below:

```
data three;
merge one two;
```

```
by id;
run;
```

Once the merge is complete, let's run a PROC PRINT to check the results:

```
proc print data=three;
run;
```

Obs	id	name	age
1	1	Roberd	15

The results show that the second dataset listed in the MERGE statement overwrites the data read from the first dataset, particularly when both datasets share the same variable. Let's look at why this happens. Once the compilation phase is complete, the execution phase reads and writes data from the first dataset to the final output. However, when SAS then reads the second dataset (since the merge is performed by id, and the id values match), the name value of "Robert" is overwritten by the value "Roberd" from the second dataset, and it's that overwritten value that gets written to the output.

So, if we want a particular dataset's value to remain untouched, we should list that dataset last in the MERGE statement, since nothing can overwrite it after that point. This is shown below, where we don't want the value "Robert" from dataset one to be overwritten, so the name variable is preserved:

```
data three_modified;
merge two one;
by id;
run;
```

Let's run PROC PRINT again to check the results:

```
proc print data=three_modified;
run;
```

Obs	id	name	age
1	1	Robert	15

While reordering the datasets gets us the result we want, this approach can become difficult when both datasets share many common variables. A better approach is to ensure only one instance of each shared variable exists in the MERGE statement, so nothing gets overwritten in the first place. This is best achieved using a DROP statement on the duplicate variable in whichever dataset we don't want it read from. That way, we can keep the original order of datasets one and twoin the MERGE statement, and since we're not interested in the name value from dataset two, we simply drop it there, as shown below:

```
data three_modified;
merge one two (drop=name);
by id;
run;
```

Now name is read only from dataset one, and since it's dropped from dataset two, there's nothing left to overwrite it. The original value is written to the final output, three_modified:

```
proc print data=three_modified;
run;
```

Obs	id	name	age
1	1	Robert	15

Now that we've examined compilation and execution in depth, understanding these phases at a deeper level allows us to avoid unexpected results. It's time to move on to _N_.

Final topic: _N_ is an iteration count, not an observation number

_N_ is an automatic, temporary variable created during DATA step execution and is only available during that step. It represents the iteration count of the DATA step. To understand what "iteration" means, let's look at a simple DATA step reading data from DATALINES. There are a few important details worth examining here, since a lot happens during DATA step execution.

```
data _null_;
input var;
datalines;
10
20
30
;
run;
```

In this DATA step, once execution begins, _N_ starts with a default value of 1 as soon as SAS reaches the DATA statement. The INPUT statement then reads a record from DATALINES, and control returns to the top of the DATA step. This is where SAS increments _N_ by 1, moving its value to 2. The INPUT statement reads the next record, control returns to the DATAstatement again, and _N_ increments to 3. This continues until SAS tries to loop back once more but finds no more records to read, so _N_ stops at 3.

We can use a PUT statement after the INPUT statement to write the values of var and _N_ to the log:

```
data _null_;
input var;
 put var = _n_=;
datalines;
10
20
```

```
30
;
run;
```

This produces the following in the log:

```
var=10 _N_=1
var=20 _N_=2
var=30 _N_=3
```

Since _N_ increases by one with each record, it's tempting for a less experienced SAS programmer to assume this is the observation number. That assumption isn't accurate. To see why, let's modify the same DATA step to read the data lines inside a DO loop instead, and again use a PUT statement to track the value of _N_:

```
data _null_;
do i=1 to 3;
input var;
put var=_n_=;
end;
datalines;
10
20
30
;
run;
```

This time, the log shows:

```
var=10 _N_=1
var=20 _N_=1
```

```
var=30 _N_=1
```

Even though the same three records are read, _N_ stays at 1 throughout. This is because the DATA step "iterates" only when execution loops back to the top of the DATA step. When data is read inside a DO loop, control never returns to the DATA statement between reads, so _N_ doesn't increment. The data is read within the loop but isn't written out until the end of the step, unlike the earlier example, where each record triggers a new iteration.

This distinction matters. If we try to subset data using _N_, we can get very different results depending on how the data is read. For example, using if _N_=1 produces different outcomes depending on whether the data is read with a simple INPUT statement or inside a DO loop.

Let's look at the first case:

```
data test0;
input var;
 if _n_=1;
datalines;
10
20
30
;
run;
```

This produces only one observation, as shown by PROC PRINT:

Obs	var
1	10

Now let's subset the data using if _N_=1 in the DO loop version, where _N_ stays at 1 for every record:

```
data test1 (drop = i);
do i=1 to 3;
input var;
if _n_=1 then
output;
end;
datalines;
10
20
30
;
run;
```

Running PROC PRINT on this result shows a different outcome: since _N_ remains 1 for all three records, all three are output, clearly demonstrating that _N_ is not the observation number:

Obs	var
1	10
2	20
3	30

Conclusion

To conclude, a deeper understanding of the DATA step is essential to avoid unexpected results. A few key takeaways:

- Compilation always runs first and sets variable attributes; execution then reads and writes the data.
- Compilation sets attributes based on the first instance it encounters, so if a specific attribute is required, the code must be structured so SAS's compiler sees the desired attribute first.
- When multiple datasets share the same variable, typically in a MERGE or UPDATE statement, that variable's value will be overwritten by the later-listed dataset. If this isn't the desired outcome, it needs to be planned for, and this paper discussed a few strategies to prevent it.

- Finally, _N_ reflects the iteration count of the DATA step, not the observation number, a distinction that matters especially when subsetting data using _N_.

Contact Information

Your comments and questions are valued and encouraged. Contact the author at:

Kiran Venna

Email: kvenna.sas@gmail.com

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ®indicates USA registration.